

FUNDAMENTOS DE SISTEMAS OPERATIVOS 2026

FACULTAD DE INGENIERÍA — UNIVERSIDAD NACIONAL DE MAR DEL PLATA

Trabajo Práctico Especial

Evaluación de Productos

Zephyr OS vs MOSIX

Integrantes:

ARRIAGA Mario E., BELLONE Martín, BISCAY Federico J., CALLA ALIENDE Federico,
CARDOZO Lucas

Fecha de presentación: 3 de Junio de 2026 · 13:30 hs

Índice

1. Introducción	3
2. Origen y Respaldo	3
2.1. Zephyr OS — Linux Foundation	3
2.2. MOSIX — Universidad Hebrea de Jerusalén	3
3. Características Generales	4
3.1. Zephyr OS	4
3.2. MOSIX	4
4. Sistema de Archivos	5
4.1. Zephyr OS	5
4.2. MOSIX	5
5. Administración de Memoria	6
5.1. Zephyr OS	6
5.2. MOSIX	6
6. Administración del Procesador	7
6.1. Zephyr OS	7
6.2. MOSIX	7
7. Seguridad	8
7.1. Zephyr OS	8
7.2. MOSIX	8
8. Facilidades para Desarrolladores	9
8.1. Zephyr OS	9
8.2. MOSIX	9
9. Puntos Fuertes y Debiles Frente a la Competencia	10
9.1. Zephyr OS	10
9.2. MOSIX	10
10. Presencia en el Mercado y Soporte	11
10.1. Adopción y comunidad	11
11. Comparativa Técnica Completa	12
12. Costos	12
12.1. Zephyr OS	12
12.2. Mosix	12
13. Conclusiones y Recomendaciones	13
13.1. Zephyr OS — Conclusión	13
13.2. MOSIX — Conclusión	13
13.3. Matriz de Decisión	13
14. Glosario de Términos Técnicos	14
15. Bibliografía	15

1. Introducción

En este informe comparamos dos sistemas operativos con propósitos muy diferentes. El objetivo es entender qué hace cada uno, para qué sirve, y cuál conviene usar según el contexto. **Los dos sistemas que analizamos son:**

- **Zephyr OS:** RTOS (Real-Time Operating System) orientado a dispositivos embebidos con restricciones severas de recursos, tales como sensores IoT, dispositivos vestibles (wearables) o equipos industriales. Mantenido por la Linux Foundation desde 2016 bajo licencia Apache 2.0.
- **MOSIX:** sistema de clustering basado en el paradigma SSI (Single System Image), cuyo objetivo es agregar múltiples nodos de computación y presentarlos como un único sistema lógico. Desarrollado en la Hebrew University of Jerusalem bajo la dirección del Prof. Amnon Barak (1977), migró a Linux en 1999 y fue descontinuado en octubre de 2017.

Aunque estos dos sistemas no compiten entre sí (resuelven problemas distintos), compararlos ayuda a entender cómo diferentes necesidades generan soluciones arquitectónicas radicalmente opuestas.

2. Origen y Respaldo

2.1. Zephyr OS — Linux Foundation

Zephyr OS nació en febrero de 2016 cuando la empresa Wind River donó su kernel (núcleo del SO) a la Linux Foundation, la misma organización que cuida el kernel de Linux.

Hoy cuenta con el respaldo de empresas como Qualcomm, Nordic, Google, Intel, NXP, Espressif (los fabricantes del ESP32) y Arduino, entre otros. Tiene más de 3.000 personas contribuyendo con código en más de 70 países.

2.2. MOSIX — Universidad Hebrea de Jerusalén

MOSIX fue creado por el Profesor Amnon Barak en 1977. Comenzó con computadoras PDP-11 antiguas y evolucionó hasta funcionar sobre Linux a partir de 1999. En 2017 salió su última versión (4.4.4) y desde entonces no tiene actualizaciones ni soporte.

Es un proyecto académico con valor histórico: demostró conceptos que hoy usan tecnologías modernas como Kubernetes, pero no está pensado para uso en producción actual.

Aspecto	Zephyr OS	MOSIX
Organización	Linux Foundation	Universidad Hebrea de Jerusalén
Año de inicio	2016	1977
Tipo	Código abierto (Apache 2.0)	Propietario y restrictivo
Segmento de mercado	Dispositivos pequeños (IoT)	Clusters de computadoras
Estado actual	Activo (versión LTS3, 2026)	Inactivo desde 2017

3. Características Generales

3.1. Zephyr OS

Zephyr tiene un kernel monolítico (todo el núcleo compilado en un solo bloque), pero muy configurable: se puede elegir exactamente qué partes incluir para ocupar el mínimo de memoria posible. Puede correr en dispositivos con unos pocos KBs de RAM.

Usa un modelo llamado **Single Address Space**: el kernel y las aplicaciones comparten el mismo espacio de memoria. Las llamadas al sistema son simples llamadas a funciones C, sin el costo de cambiar de modo privilegiado. Esto lo hace muy rápido y predecible.

Está diseñado para responder a eventos externos de forma predecible y casi instantánea, algo vital en sensores industriales o dispositivos médicos.

3.2. MOSIX

MOSIX no es un SO independiente; es una capa adicional que se instala sobre Linux. Su función principal es hacer que un grupo de computadoras parezca un sistema con muchos procesadores.

Cada nodo monitorea constantemente sus propios recursos (uso de CPU, memoria libre, etc.) y comparte de forma aleatoria y regular esta información con los demás para mantener al cluster autoorganizado.

Su característica más innovadora es la **migración preemptiva de procesos**: si una computadora del cluster está muy ocupada, MOSIX puede mover un proceso en plena ejecución a otra computadora con más recursos, de forma completamente transparente para el programa.

4. Sistema de Archivos

4.1. Zephyr OS

Zephyr usa una capa de abstracción llamada VFS (Virtual File System) que permite trabajar con distintos tipos de almacenamiento usando la misma interfaz. Soporta tres sistemas de archivos principales:

- **LittleFS:** diseñado para memorias flash (como las que usan los microcontroladores). Es resistente a cortes de energía abruptos y gestiona automáticamente el desgaste de la memoria.
- **FAT FS:** el mismo formato que usan los pendrives USB y tarjetas SD. Fácil de leer desde una PC, pero menos robusto ante fallos de energía.
- **NVS (Non-Volatile Storage):** almacenamiento simple de tipo clave-valor (como un diccionario) para guardar configuraciones del dispositivo.

Limitaciones: no soporta permisos de archivo (cualquier proceso puede leer/escribir todo) ni enlaces simbólicos. Es suficiente para dispositivos embebidos, pero no para sistemas multi-usuario.

4.2. MOSIX

MOSIX implementa MFS (MOSIX File System), un sistema de archivos que provee una visión unificada de todos los archivos montados en cada nodo del clúster. MFS es una capa instalada sobre el sistema de archivos de cada computadora, lo que lo hace genérico y escalable.

5. Administración de Memoria

5.1. Zephyr OS

Para proteger las zonas de memoria, Zephyr usa la MPU (en lugar de la MMU que usan las PCs). La MPU divide la RAM en regiones y define qué puede hacer cada zona: leer, escribir o ejecutar código. Si un programa intenta acceder a una zona no permitida, el hardware lanza una excepción de inmediato.

El swap está desactivado por defecto (intercambio de memoria con disco) en estos dispositivos. Todos los recursos de memoria se definen en tiempo de compilación, lo que hace el sistema muy predecible pero también requiere planificación cuidadosa. Para «ampliar» la memoria del sistema debe activarse explícitamente la paginación por demanda.

5.2. MOSIX

Cada nodo del cluster tiene su memoria RAM propia e independiente. No existe memoria compartida entre nodos. Cuando un nodo empieza a quedarse sin RAM, MOSIX activa el mecanismo «Memory Ushering»:

Memory Ushering (prevención de agotado): MOSIX monitorea continuamente la frecuencia de paginación. Si detecta que este valor es excesivo, intenta mover procesos enteros a otro nodo con más espacio disponible, y otros factores que lo hagan el más apto para recibir ese proceso.

Limitación importante: mover un proceso de una computadora a otra implica transferir por red todos sus datos en RAM (código, variables, pila de llamadas). Para procesos que usan varios gigabytes de memoria, esto puede tardar minutos, lo cual es inaceptable en muchos contextos.

Aspecto	Zephyr OS	MOSIX
Hardware objetivo	Microcontroladores (KB de RAM)	Clusters de PCs (GB de RAM por nodo)
Protección de memoria	MPU (regiones con permisos)	Aislamiento por nodo (Linux nativo)
Memoria virtual	Solo en modelos avanzados con MMU	No existe entre nodos
¿Tiene swap?	Por defecto no (recursos fijos en compilación)	No (usa migración de procesos)
¿Cuándo se define la memoria?	Por defecto en compilación (estático)	En tiempo de ejecución (dinámico)

6. Administración del Procesador

6.1. Zephyr OS

Zephyr usa un scheduler basado en prioridades fijas: cada thread (hilo de ejecución) tiene un número de prioridad. El thread con mayor prioridad siempre es el primero en usar la CPU. Si la prioridad del hilo es representada con un número negativo, significa que se ejecutará en el modo cooperativo.

- **Modo cooperativo:** el thread decide voluntariamente cuándo ceder la CPU. Útil para tareas que deben terminar sin interrupciones.
- **Modo preemptivo:** el SO puede interrumpir un thread en cualquier momento si llega otro de mayor prioridad. Garantiza que las tareas críticas respondan siempre a tiempo.

Sin time-slicing: no hay un reloj que distribuya tiempo de CPU en rodajas iguales (como lo hace Linux). Esto da mayor determinismo (comportamiento predecible) pero puede causar starvation: un thread de baja prioridad nunca llega a ejecutarse si siempre hay uno de mayor prioridad esperando.

Priority Inheritance: si un thread de baja prioridad bloquea a uno de alta prioridad, Zephyr eleva temporalmente la prioridad del thread bloqueante. Esto evita la inversión de prioridad, un bug clásico que causó, por ejemplo, el reinicio del rover Pathfinder en Marte (1997).

6.2. MOSIX

MOSIX no implementa una administración del procesador. Esa tarea es responsabilidad del kernel de Linux corriendo por debajo. MOSIX decide en qué nodo del clúster se ejecutará el proceso a ser migrado, el cuál fue seleccionado por el algoritmo de administración de memoria.

Para tomar esa decisión, evalúa múltiples factores simultáneamente: velocidad de CPU, carga actual, memoria disponible, latencia de red y cantidad de núcleos, exigencia del proceso, entre otros factores. Es un balanceo de carga multiparamétrico.

Tiene tres niveles de Scheduling:

- **Largo plazo:** dónde colocar un proceso cuando comienza.
- **Medio plazo:** activar Memory Ushering cuando hay escasez de memoria.
- **Corto plazo:** evaluar continuamente si conviene mover un proceso a otro nodo.

SSI (Single System Image): desde el punto de vista del usuario, el cluster entero se ve como una sola computadora con muchos procesadores. Los programas no necesitan ser modificados para aprovechar el cluster; MOSIX se encarga de distribuirlos de forma transparente.

Aspecto	Zephyr OS	MOSIX
Unidad que se planifica	Threads (livianos)	Procesos completos (pesados)
Niveles de scheduler	Solo corto plazo	Largo + medio + corto plazo
Algoritmo	Prioridad fija sin time-slicing	Balanceo multi-paramétrico
Objetivo principal	Mínima latencia, determinismo	Máximo throughput del cluster

7. Seguridad

7.1. Zephyr OS

Zephyr tiene un modelo de seguridad pensado para dispositivos IoT que se conectan a internet y deben seguir regulaciones estrictas (médicas, industriales, automotrices).

- **Aislamiento por MPU:** el código del kernel es de solo lectura; la RAM de usuario no puede ejecutarse como código. Esto previene muchos ataques comunes.
- **ARM TrustZone:** en chips compatibles, divide el chip en dos mundos: uno seguro (para claves criptográficas) y uno normal (para aplicaciones). Las claves nunca abandonan el mundo seguro.
- **Arranque seguro (Secure Boot):** al encender el dispositivo, cada etapa del arranque verifica la firma digital de la siguiente. Si alguien adultera el firmware, el dispositivo lo detecta y no arranca.
- **TLS 1.2+:** comunicaciones cifradas mínimo con TLS 1.2 (versiones anteriores, con vulnerabilidades conocidas, están bloqueadas).

OpenSSF Gold Badge: es una certificación independiente de seguridad. Para obtenerlo, el proyecto debe tener cobertura de tests del 90%, revisión de código por dos personas, y pasar una auditoría externa. Zephyr lo obtuvo en 2018. Esta certificación reduce significativamente el costo de auditorías regulatorias.

7.2. MOSIX

Advertencia importante: MOSIX no tiene soporte de seguridad activo desde 2017. Vulnerabilidades conocidas desde entonces no han sido corregidas.

MOSIX requiere que todas las computadoras del cluster confíen mutuamente entre sí. No hay mecanismo para verificar si un nodo fue comprometido. Si un atacante toma control de una computadora del cluster, tiene acceso potencial a todos los procesos que corran en él.

Por otro lado MOSIX sí tiene un mecanismo para asegurar que los procesos solo puedan ejecutar system calls en el nodo de origen. Un proceso migrado está obligado a ejecutar tales llamadas en el nodo original.

Problema grave: los archivos de checkpoint (donde se guarda el estado completo de un proceso para migrarlo) no están cifrados por defecto. Esto significa que contraseñas, claves de API y datos sensibles que estén en la memoria del proceso quedan expuestos en texto claro.

Aspecto	Zephyr OS	MOSIX
Aislamiento	Hardware (MPU + TrustZone)	Lógico (módulo de kernel)
Criptografía	PSA Crypto + mbedTLS integrados	No disponible
Arranque seguro	Sí, con firmas asimétricas	No disponible
Certificaciones	OpenSSF Gold Badge (2018)	No aplica
Multi-tenant	Compatible	Incompatible (riesgo crítico)
Parches de seguridad	Sí (soporte LTS activo)	Ninguno desde 2017

8. Facilidades para Desarrolladores

8.1. Zephyr OS

Zephyr tiene un ecosistema de herramientas completo y bien documentado:

- **West:** herramienta de línea de comandos que gestiona todo el proyecto: descargar dependencias, compilar, flashear el firmware al dispositivo y depurar errores.
- **CMake + KConfig:** sistema de compilación (el mismo que usa Linux) con un sistema de configuración por menús donde se activan/desactivan componentes fácilmente.
- **Device Tree:** archivo de texto que describe el hardware del dispositivo (qué periféricos hay, en qué pines, con qué IRQ). El mismo driver funciona en diferentes chips sin modificar código.
- **QEMU:** permite emular el dispositivo en la PC antes de tener el hardware real. Ideal para desarrollo y pruebas.
- **API estilo POSIX:** funciones similares a las de Linux estándar (open, read, write, socket...), lo que facilita la transición de programadores que vienen de Linux.

La documentación en docs.zephyrproject.org es extensa, actualizada y cubre desde guías de inicio hasta referencias de API completas. Hay comunidad activa en Discord y GitHub.

8.2. MOSIX

La gran ventaja de MOSIX para desarrolladores es su transparencia total: los programas existentes no necesitan ser modificados ni recompilados para funcionar en el cluster.

- **mosrun:** comando que marca un proceso como “migrable”. El scheduler decide automáticamente si conviene moverlo a otro nodo.
- **mosmon:** monitor en tiempo real que muestra el estado del cluster.
- **mosps / mostat:** equivalentes a ps y stat pero para todo el cluster.

Limitaciones críticas para desarrolladores: MOSIX no soporta threads POSIX (la forma estándar de hacer multithreading hoy en día) ni memoria compartida entre procesos de distintos nodos. Programas modernos que usen estas características no pueden migrarse.

9. Puntos Fuertes y Debiles Frente a la Competencia

9.1. Zephyr OS

Al evaluar Zephyr OS frente a sus competidores directos en el ámbito de los sistemas operativos de tiempo real (como FreeRTOS o NuttX), se destacan ventajas e inconvenientes muy claros debido a su diseño moderno y su filosofía de desarrollo.

- **Puntos Fuertes:** Su principal ventaja es que ofrece un ecosistema integrado y modular. A diferencia de FreeRTOS, que suele requerir que el desarrollador busque y configure por separado las pilas de red o conectividad inalámbrica, Zephyr incluye soporte nativo para BLE, Wi-Fi y Thread dentro del propio kernel. Además, su gobernanza neutral bajo la Linux Foundation garantiza que ninguna corporación pueda discontinuar el proyecto de manera unilateral, eliminando el riesgo de dependencia del proveedor (vendor lock-in). Por último, gracias al uso de Device Tree, la abstracción de hardware es avanzada, lo que permite migrar un desarrollo de un chip a otro con mínimos cambios en el código.

- **Puntos Débiles:** El contraflujo de su potencia es una mayor curva de aprendizaje. La suite de herramientas basada en West, CMake y KConfig imita la complejidad del kernel Linux, representando una barrera de entrada más alta para equipos acostumbrados a entornos de desarrollo más simples o lineales. Asimismo, presenta un mayor consumo de recursos base; aunque su footprint mínimo es muy bajo, un kernel de Zephyr con configuraciones básicas suele consumir ligeramente más memoria que un microkernel minimalista como FreeRTOS puro.

9.2. MOSIX

El análisis de MOSIX frente a competidores del cómputo de alto rendimiento (HPC) y la orquestación moderna, como SLURM o Kubernetes, permite entender por qué una tecnología tan innovadora quedó relegada al ámbito estrictamente histórico.

- **Puntos Fuertes:** Su mayor hito conceptual es la transparencia total para el usuario a través de su arquitectura de Imagen de Sistema Único (SSI). El programador no necesita modificar el código de sus programas ni usar librerías complejas (como MPI) para aprovechar el cluster; el sistema operativo distribuye la carga por sí solo. A esto se suma su capacidad de migración dinámica y proactiva: el algoritmo de Memory Ushering y la capacidad de mover procesos enteros en plena ejecución según la carga de cada computadora fueron soluciones sumamente avanzadas para su época.

- **Puntos Débiles:** El punto más crítico es su abandono y obsolescencia. Al no recibir parches ni actualizaciones desde 2017, carece por completo de protección contra vulnerabilidades arquitectónicas modernas del hardware (como Spectre y Meltdown), lo que vuelve su uso actual un riesgo inaceptable. Técnicamente, presenta una incompatibilidad severa con el software moderno debido a la falta de soporte para hilos (threads) POSIX y memoria compartida entre nodos, lo que lo descalifica para ejecutar casi cualquier aplicación concurrente actual. Por último, su modelo de licenciamiento propietario y cerrado adoptado en 2001 ahuyentó a la comunidad de código abierto, acelerando su caída frente a alternativas libres, flexibles y estandarizadas como Kubernetes.

10. Presencia en el Mercado y Soporte

10.1. Adopción y comunidad

Aspecto	Zephyr OS	MOSIX
Contribuyentes activos	3.000+ en 70+ países	0 (abandonado)
Plataformas soportadas	1.000+ boards (ARM, RISC-V, x86...)	Clusters Linux históricos
Respaldo corporativo	Intel, Nordic, Google, Renesas, NXP...	Ninguno
Documentación	Completa y actualizada (2026)	Desactualizada desde 2017
Soporte comercial	Disponible via Wind River y miembros	No disponible
Seguridad (parches)	Activo (LTS con backports)	Sin parches desde +8 años

Productos reales que usan Zephyr OS:

- Audífonos Oticon More (dispositivo médico)
- Laptops Framework (controlador de teclado)
- Google Chromebook (controlador embebido)
- Sensores de irrigación GARDENA
- Turbinas eólicas Vestas

MOSIX no registra casos de uso modernos. Su uso activo se limitó a clusters universitarios entre aproximadamente 1999 y 2010.

11. Comparativa Técnica Completa

Nota importante: Zephyr OS y MOSIX no son competidores directos. Resuelven problemas completamente diferentes. Esta comparación tiene valor pedagógico para entender cómo el dominio de aplicación define la arquitectura del SO.

Característica	Zephyr OS	MOSIX
Tipo	RTOS embebido	Extensión de cluster sobre Linux
¿Para qué sirve?	Dispositivos IoT con poca memoria	Distribuir procesos en un cluster
Licencia	Apache 2.0 (abierta y permisiva)	Propietaria restrictiva
RAM mínima requerida	Unos pocos KB	N/A (requiere PCs completas)
Arquitectura del kernel	Monolítico configurable	Módulo sobre kernel Linux
Sistema de archivos	LittleFS, FAT FS, NVS (propios)	MFS (cluster) + DFSA (optimización)
Gestión de memoria	MPU + Memory Domains	Shared-nothing + Memory Ushe-ring
Scheduling	Prioridad (coop/preemptive)	Migración entre nodos del cluster
¿Es tiempo real?	Sí (determinístico)	No
Migración de procesos	No aplica	Sí, transparente y preemptiva
Seguridad	MPU + TrustZone + PSA Crypto	Sandboxing básico (sin crypto)
Herramientas	West, CMake, KConfig, QEMU, DevTree	mosrun, mosmon, mosps, mostat
Comunidad	Muy activa (3.000+ contribuyentes)	Inactiva desde oct 2017
Última versión	LTS3 (2026)	4.4.4 (octubre 2017)
Competidores directos	FreeRTOS, NuttX, RIOT OS	SLURM, Kubernetes, OpenMPI

12. Costos

12.1. Zephyr OS

Zephyr OS no tiene ningún costo económico. Es un sistema operativo completamente gratuito y de código abierto.

12.2. Mosix

En sus comienzos, el sistema fue completamente gratuito y de libre uso. Pero alrededor del año 2001, el código se volvió cerrado y se empezó a cobrar una licencia comercial.

13. Conclusiones y Recomendaciones

13.1. Zephyr OS — Conclusión

Zephyr OS es la opción sólida y moderna para proyectos IoT embebidos en 2026. Sus puntos fuertes desde la perspectiva de consultoría son:

- **Gobernanza neutral:** al estar bajo la Linux Foundation (con múltiples sponsors corporativos), ninguna empresa puede discontinuarlo unilateralmente. Esto elimina el “vendor lock-in” que tienen alternativas como FreeRTOS (Amazon) o ThreadX (Microsoft).
- **Seguridad certificada:** el OpenSSF Gold Badge y la auditoría de NCC Group (2020) son validaciones externas que reducen el costo de auditorías regulatorias para productos médicos e industriales.
- **Soporte a largo plazo:** los productos IoT industriales y médicos deben durar 10-20 años. La versión LTS3 garantiza parches de seguridad durante ese período, evitando rediseños de hardware costosos.
- **Ecosistema de conectividad completo:** BLE, Wi-Fi, Thread, LoRa, Cellular, CAN... todo integrado en el kernel, sin necesidad de drivers propietarios adicionales.

13.2. MOSIX — Conclusión

MOSIX tiene valor histórico e interés académico, pero no debe usarse en producción. Sus contribuciones conceptuales fueron importantes: fue pionero en migración preemptiva de procesos (1977) y en el concepto de Single System Image (cluster que parece una sola máquina). Estos conceptos influyeron en lo que hoy es Kubernetes.

Sus problemas en 2026 son inaceptables para producción:

- Sin parches de seguridad desde hace más de 8 años.
- Vulnerabilidades tipo Spectre/Meltdown sin corrección posible (código propietario, desarrollador inactivo).
- Archivos de checkpoint sin cifrado: contraseñas y claves en texto claro.
- Licencia propietaria que impide auditar o modificar el código.
- Sin soporte para threads POSIX ni memoria compartida, incompatible con software moderno.

13.3. Matriz de Decisión

Si necesitas...	Recomendación
Producto IoT comercial en 2026	Zephyr OS
Seguridad robusta + conectividad wireless	Zephyr OS
Producto industrial/médico con ciclo de vida largo	Zephyr OS (LTS3)
Portabilidad entre distintos chips/vendedores	Zephyr OS
Prototipo rápido, equipo sin experiencia en RTOS	FreeRTOS
Aprender conceptos de clustering histórico	MOSIX (solo estudio)
HPC real en producción en 2026	SLURM o Kubernetes

«No existe el mejor sistema operativo — existe el correcto para tu problema.»

14. Glosario de Términos Técnicos

- **AMP (Asymmetric Multi-Processing):** uso de múltiples procesadores donde cada uno ejecuta un SO o tarea diferente.
- **Apache 2.0:** licencia de software libre que permite usar, modificar y distribuir el código, incluso en productos comerciales, sin pagar regalías.
- **ARM TrustZone:** tecnología de seguridad de chips ARM que divide el procesador en dos mundos aislados: uno seguro (para claves y criptografía) y uno normal.
- **BLE (Bluetooth Low Energy):** versión de Bluetooth de muy bajo consumo energético, usada en wearables e IoT.
- **CAN bus:** protocolo de comunicación muy usado en automóviles e industria.
- **Checkpoint/Restart:** mecanismo que guarda el estado completo de un proceso (memoria, registros, descriptores) para poder restaurarlo más tarde, posiblemente en otra máquina.
- **CMake:** herramienta que genera archivos de compilación para distintas plataformas.
- **Cluster:** conjunto de computadoras interconectadas que trabajan como una sola unidad.
- **DFSA (Direct File System Access):** capa de MOSIX que redirige las peticiones de archivos al nodo del cluster donde está el archivo.
- **Device Tree:** archivo de texto que describe el hardware de un dispositivo embebido (pines, IRQs, periféricos).
- **FAT FS:** sistema de archivos simple, compatible con Windows, usado en pendrives y tarjetas SD.
- **HPC (High Performance Computing):** cómputo de alto rendimiento; uso de clusters para resolver problemas científicos muy costosos computacionalmente.
- **IoT (Internet of Things):** dispositivos físicos conectados a internet: sensores, actuadores, electrodomésticos inteligentes, etc.
- **IRQ (Interrupt Request):** señal de hardware que avisa al procesador que un periférico necesita atención.
- **KConfig:** sistema de configuración por menús, permite activar/desactivar componentes del SO.
- **LittleFS:** sistema de archivos para memorias flash, resistente a fallos de energía.
- **LoRa:** tecnología de comunicación inalámbrica de largo alcance y muy bajo consumo, usada en IoT.
- **LTS (Long-Term Support):** versión con soporte extendido y parches garantizados durante años.
- **Memory Ushering:** mecanismo de MOSIX que migra procesos proactivamente para evitar que un nodo se quede sin RAM.
- **MMU (Memory Management Unit):** hardware que traduce direcciones lógicas a físicas y permite memoria virtual.
- **MPU (Memory Protection Unit):** hardware más simple que la MMU; define zonas de RAM con permisos de lectura/escritura/ejecución.
- **mutex:** mecanismo de sincronización que garantiza que solo un thread acceda a un recurso a la vez.
- **NVS (Non-Volatile Storage):** almacenamiento clave-valor en flash para configuraciones del dispositivo.
- **OpenSSF Gold Badge:** certificación de seguridad de la Open Source Security Foundation que valida buenas prácticas de desarrollo seguro.
- **POSIX:** estándar que define la API de sistemas operativos tipo Unix; garantiza compatibilidad entre sistemas.
- **PSA Crypto API:** interfaz estándar de Arm para operaciones criptográficas en dispositivos embebidos.
- **RTOS (Real-Time Operating System):** SO que garantiza respuesta en tiempos predecibles; crítico en sistemas médicos, industriales y automotrices.
- **Scheduler:** componente del SO que decide qué proceso/thread usa la CPU en cada momento.
- **SLURM:** sistema de gestión de trabajos para clusters HPC; el más usado en supercomputadoras modernas.

- **SSI (Single System Image):** ilusión de que un cluster completo es una sola máquina; los procesos se distribuyen transparentemente.
- **Starvation:** situación donde un proceso nunca llega a ejecutarse porque siempre hay otros con mayor prioridad.
- **Thread (hilo):** subtarea dentro de un proceso; más liviana y rápida de crear que un proceso completo.
- **TLS (Transport Layer Security):** protocolo criptográfico que protege las comunicaciones por red (HTTPS lo usa).
- **VFS (Virtual File System):** capa de abstracción que permite usar distintos sistemas de archivos con la misma interfaz.
- **West:** herramienta de línea de comandos oficial de Zephyr para gestionar, compilar y flashear proyectos.

15. Bibliografía

- Zephyr Project Documentation: <https://docs.zephyrproject.org/>
- Zephyr GitHub Repository: <https://github.com/zephyrproject-rtos/zephyr>
- Linux Foundation Research «Zephyr at 10» (marzo 2026): <https://www.linuxfoundation.org/research/zephyr-turns-10>
- OpenSSF Best Practices Gold Badge: <https://www.bestpractices.dev/projects/74/gold>
- MOSIX Official Site: <http://www.mosix.org/>
- The MOSIX Cluster Management System for Parallel Computing on Linux Clusters and Multi-Cluster Private Clouds - White Paper
- The MOSIX Direct File System Access Method for Supporting Scalable Cluster - Lior Amar, Amnon Barak and Amnon Shiloh, Institute of Computer Science, The Hebrew University of Jerusalem, 91904 Israel
- MOSIX Distributions & Licensing: https://mosix.cs.huji.ac.il/txt_distributions.html
- PSA Crypto API Documentation (Arm): <https://developer.arm.com/architectures/security-architectures/platform-security-architecture>
- SLURM Workload Manager: <https://slurm.schedmd.com/>
- Top500 Supercomputers List: <https://www.top500.org/>
- Linux Foundation Members List (2025-2026): <https://www.linuxfoundation.org/members>
- <https://www.intel.com/content/www/us/en/developer/articles/community/zephyr-story-how-became-self-sustaining-ecosystem.html>
- <https://www.youtube.com/watch?v=6PpjYa1kJ1U>

Documento elaborado para el Trabajo Práctico Especial de Fundamentos de Sistemas Operativos
Universidad Nacional de Mar del Plata · Mayo 2026